Blug Forum

Dokumentation

REST-backend med PostgreSQL
Windows-start, API, tester och överlämning
Säkerhets- och integritetsanalys

Export av projektets två befintliga dokument
Verifieringsdatum: 12 oktober 2026

Lärarens exakta endpointkontrakt och Blugs faktiska frontend återstår att verifiera. Se dokumentens kända begränsningar.

Blug Forum – fortsättningsguide

Det befintliga projektet används: Java 25, Spring Boot 4.0.5, Gradle wrapper 9.4.1 och PostgreSQL 18. Ingen ny frontend eller projektstruktur har införts. `UserData` och dess repository finns kvar men används inte för autentisering. Användare, profiler och behörigheter använder tabellen `users`.

Start i Windows PowerShell

Installera inte om fungerande program. Förutsättningar: JDK 25 på PATH, Git och Docker Desktop med Linux-motorn igång. Kör i den befintliga katalogen:

```
Set-Location C:\Users\Angelica\code\blug-forum
java -version
docker compose up -d postgres
docker compose ps
.\gradlew.bat bootRun
```

API: `http://localhost:8080/api`; hälsa: `/actuator/health`; befintlig dashboard: `/dashboard`. Compose och standardinställningen använder båda `blug_forum`. Om den befintliga databasen heter `forum`, använd före start:

```
$env:SPRING_DATASOURCE_URL = 'jdbc:postgresql://localhost:5432/forum'
```

Radera aldrig volymen för att lösa konfigurationsfel. `ddl-auto: update` återanvänder schemat. Nya blockkolumner får standardvärdet false även för befintliga rader. Detta är ingen produktionsstrategi för schemaversionering.

Test

Integrationstesterna använder en separat lokal PostgreSQL-databas och syntetiska uppgifter; datamutationer i MockMvc-tester rullas tillbaka. Skapa databasen en gång (om den redan finns, återanvänd den):

```
docker compose exec -T postgres createdb -U postgres blug_forum_test
.\gradlew.bat build
```

Resultat: `build/reports/tests/test/index.html`. Kör inte mutationstester med en datasource som pekar på gruppens riktiga data. Live-HTTP-verifiering och återläsning använder också isolerade databaser.

För att upprepa livekontrollen: starta en separat process på 8081 med syntetisk administratör i testdatabasen. Använd en ny administratörsidentitet om det befintliga kontots lösenord är okänt; provisioneringen skriver inte över ett konto.

```
$env:BLUG_ADMIN_USERNAME = 'smoke-admin'
$env:BLUG_ADMIN_EMAIL = 'smoke-admin@example.test'
$taskSmokeSecret = Read-Host 'Syntetiskt administratörslösenord' -AsSecureString
$env:BLUG_ADMIN_PASSWORD = [System.Net.NetworkCredential]::new('', $taskSmokeSecret).Password
java -jar build/libs/blug-forum-0.0.1-SNAPSHOT.jar
--spring.datasource.url=jdbc:postgresql://localhost:5432/blug_forum_test --server.port=8081
```

I en andra PowerShell-terminal i projektkatalogen:

```
$taskSmokeSecret = Read-Host 'Samma syntetiska administratörslösenord' -AsSecureString
.\scripts\smoke-test.ps1 -ConfirmIsolatedDatabase -AdminUsername smoke-admin -AdminPassword $taskSmokeSecret
```

Skriptet avbryter på fel, följer inte redirects och använder riktiga sessioncookies och CSRF-token. Det lämnar syntetiska uppgifter i testdatabasen för återläsningskontroll. `smoke-result.json` och dumpfiler ignoreras av Git. Stäng testservern med Ctrl+C efter kontrollen och rensa `Env:BLUG_ADMIN_PASSWORD` i terminalen där den sattes.

Administratör

Registrering skapar alltid USER. Inga standardkonton med `admin/admin` skapas. Valfri engångsprovisionering skriver en BCrypt-hash direkt till samma `users`-tabell som autentiseringen använder:

```
$env:BLUG_ADMIN_USERNAME = 'admin'
$env:BLUG_ADMIN_EMAIL = 'admin@example.test'
$taskAdminSecret = Read-Host 'Administratörslösenord (minst 8 tecken)' -AsSecureString
$env:BLUG_ADMIN_PASSWORD = [System.Net.NetworkCredential]::new('', $taskAdminSecret).Password
.\gradlew.bat bootRun
# Efter att processen stoppats:
Remove-Item Env:BLUG_ADMIN_PASSWORD
```

Använd egna uppgifter vid verklig drift. Lösenordet loggas inte och ska inte checkas in. Om användarnamnet redan finns ändras varken lösenord eller roll: ett befintligt USER-konto uppgraderas inte automatiskt. En behörig databasansvarig får granska och uppdatera rollen i `users` separat. Tabellen `user_data` är inte API-identitetskällan.

Session och CSRF

1. GET `/api/auth/csrf` före registrering/inloggning. Spara sessionens cookie och svarets `token` och `headerName`.
2. Skicka token i `X-CSRF-TOKEN` och samma sessioncookie vid POST, PUT och DELETE, även registrering, login och logout.
3. POST `/api/auth/login` med `application/x-www-form-urlencoded`, fälten `username` och `password`. Framgång: 200; fel lösenord: 401. Sessionens id kan bytas vid login.
4. Hämta en ny token efter login eller logout och använd den senaste sessioncookien. CSRF skyddar också anonyma skrivningar. Saknad/felaktig token: 403.
5. POST `/api/auth/logout`: 204. Profilens lösenordsbyte och radering avslutar den aktuella sessionen.

Frontendens fetch-anrop behöver `credentials: 'include'` för sessioner. CORS har inte öppnats: ingen separat frontend-origin har verifierats. Thymeleaf-formulär med `th:action` får CSRF-fält genom Spring-integrationen. Spring beskriver tokenhanteringen i CSRF-dokumentationen.

Databas och åtkomst

`forums` 1–N `forum_thread` 1–N `posts`. Trådar har en aktuell ägare i `users`; inlägg har en författare. `thread_moderators` och `thread_participants` kopplar trådar till användare. Forumets skapare är valfri efter kontoradering.

Ägare och moderator är relationer till en viss tråd, inte globala behörigheter. Den äldre enum-posten MODERATOR ger ingen extra rättighet. Privat trådinnehåll kräver ägarskap, moderatorkoppling eller deltagarkoppling; ADMIN får också läsa för tillsyn. Blockering av forum döljer underliggande trådar/inlägg; trådblockering döljer dess inlägg. Blockerade användare nekas skyddade operationer även genom äldre sessioner. Identiteten binds också till databas-id för att förhindra återanvändning av ett raderat kontos session.

Blockering behåller data för återställning. Författarens inläggsradering lämnar en dold tombstone och tar bort texten. Ägarens trådradering och administratörens DELETE är permanenta. Återställning tar bara bort blockeringen och kan inte återskapa raderad data. Vid kontoradering raderas ägda trådar inklusive deras inlägg och kontots övriga inlägg; medlemskopplingar tas bort och forumattribution sätts till null. Ägaren kan överföra en tråd innan kontot raderas.

API-kontrakt (provisoriskt tills lärarens specifikation kan läsas)

Lärarlänken kunde inte läsas: https://dm24h-tuc.lms.nodehill.se/article/blug-endpoints. Befintliga paths behålls. Kompatibilitet med Blugs frontend är inte verifierad. JSON används om inte annat anges. Id-referenser är `{ "id": 123 }`; synlighet är `{ "isPublic": false }`.

Metod och path	Begäran / beteende
GET `/api/auth/csrf`	token, headerName, parameterName
POST `/api/auth/register`	username, email, password; 201 med message
POST `/api/auth/login`	formfält username/password; 200
POST `/api/auth/logout`	204
GET `/api/forums`	lista oblockerade forum
POST `/api/forums`	name, description?, content?; medlem; 201
GET `/api/forums/{id}/threads`	id, title, isPublic, createdAt; även privata metadata
POST `/api/forums/{id}/threads`	title, content?, isPublic? (default true); 201
GET `/api/threads/{id}`	kontrollerad tråddetalj, inga deltagarlistor
PUT `/api/threads/{id}`	title, content?; aktuell ägare
DELETE `/api/threads/{id}`	aktuell ägare; 204
PUT `/api/threads/{id}/visibility`	isPublic; aktuell ägare
POST `/api/threads/{id}/transfer-ownership`	id för befintlig oblockerad användare
POST `/api/threads/{id}/moderators/add` eller `/remove`	id; aktuell ägare
POST `/api/threads/{id}/participants/add` eller `/remove`	id; ägare/trådens moderator
GET `/api/threads/{id}/posts`	endast synliga inlägg efter trådkontroll
POST `/api/threads/{id}/posts`	content; medlem med åtkomst; 201
GET `/api/posts/{id}`	synligt inlägg efter trådkontroll
PUT `/api/posts/{id}`	content; författare
DELETE `/api/posts/{id}`	författare eller trådens moderator; 204
POST `/api/posts/{id}/block`	ägare/trådens moderator; 204
GET, PUT, DELETE `/api/users/me`	egen profil; PUT email? och password?; DELETE 204
POST `/api/admin/{users,forums,threads,posts}/{id}/block` eller `/restore`	endast aktuell ADMIN
DELETE `/api/admin/{users,forums,threads,posts}/{id}`	permanent; endast ADMIN; 204

Forumsvaret: id, name, description, createdAt, blocked. Tråddetalj: id, forumId, title, content, ownerId, isPublic, createdAt, blocked. Inlägg: id, threadId, content, authorId, createdAt, updatedAt, blocked. Profil: id, username, email, role, blocked; endast egen profil/administratörens användaroperationer. Inga lösenord, hashvärden eller JPA-grafer returneras. Registrering returnerar message. Block/restore returnerar respektive DTO; DELETE och

moderatorblockering returnerar 204.

Fel: 400 validering/JSON, 401 saknad eller ogiltig identitet, 403 nekad operation/CSRF, 404 saknat eller dolt objekt, 409 datakonflikt, 500 säkert generellt meddelande. Username: 1–50 ASCII-bokstäver/siffror/`_.-`, oföränderligt i profil; email högst 254; lösenord minst 8 tecken och högst 72 UTF-8-byte; namn/titel 200; beskrivning 2000; innehåll 20000. Klientens roll, ägare, författare, id och blockfält används inte som caller-identitet.

Ägarbyte ger föregående ägare deltagaråtkomst men inte fortsatt ägarrätt. Moderatorborttagning tar bort moderatorkopplingen; eventuell separat deltagarkoppling kvarstår. Moderatorer tilldelar deltagaråtkomst direkt, utan ett extra inbjudningsflöde. En privat titel får inte innehålla hemligheter eftersom listan visar metadata. Användarnamn kan inte ändras; profilens email/lösenord kan ändras.

Att ta bort en deltagarkoppling återkallar privat medlemsåtkomst, men påverkar inte ett separat ägarskap/moderatorskap eller den allmänna åtkomsten till en offentlig tråd. Kontoblockering stoppar kontots skyddade operationer; redan publicerade inlägg blockeras separat vid behov. Restaurering av ett forum ändrar inte dess trådars eller inläggs egna blockflaggor.

Backup och återläsning

Gör aldrig återläsning över gruppens databas som ett test. Skapa en PostgreSQL custom-format dump i containern och kopiera filen utan en binär PowerShell-pipeline:

```
docker compose exec -T postgres pg_dump -U postgres -d blug_forum -Fc -f /tmp/blug-forum.dump
docker compose cp postgres:/tmp/blug-forum.dump ./blug-forum.dump
docker compose exec -T postgres createdb -U postgres blug_forum_restore_test
docker compose exec -T postgres pg_restore -U postgres -d blug_forum_restore_test --no-owner /tmp/blug-
forum.dump
docker compose exec -T postgres psql -U postgres -d blug_forum_restore_test -c 'SELECT count(*) FROM users;
SELECT count(*) FROM forums; SELECT count(*) FROM forum_thread; SELECT count(*) FROM posts;'
```

Verifiera tabeller, relationer och representativa poster; antal räcker inte ensamt. Spara dumpen skyddat utanför repo med begränsad åtkomst. Dumpen innehåller personuppgifter och hashvärden. Planerad rutin: daglig backup före ändringar, 30 dagars retention, manuell kontrollerad återläsning. Detta är en rekommendation, ingen automatisk backupfunktion. Raderingar måste återappliceras efter återläsning av äldre backup.

Gruppens nästa uppgifter

- Hämta lärarens exakta endpointkontrakt och jämför metoder, paths, begäran och svar.
- Verifiera Blugs verkliga frontend med sessioncookies och CSRF, och konfigurera endast dess origin om CORS behövs.
- Granska release-branchens behörigheter och testresultat tillsammans före skolredovisningen.
- Fastställ integritetstext, ansvarig organisation, lagringstider och backupansvar.
- För nästa sprint: loginbegränsning, paginering och granskning av HTML-rendering i den separata frontend som Blug levererar.

Verifieringsstatus 2026-10-12

- Bevarat: befintlig checkout, Java/Spring/Gradle/PostgreSQL-versioner, fungerande kompilering, `ForumThread`-namn, repositories, sessionautentisering, dashboardens utseende och gamla API-paths där möjliga. Databaser och volymer har inte raderats.
- Ursprunglig build kompilerade men 3 tester misslyckades när databasen inte körde; två äldre tester avsåg dessutom obefintliga endpoints. Det är inte samma resultat som cloud-arbetsytans tidigare fyra tester.

- Slutlig `.\gradlew.bat build`: 18 tester passerade, inga misslyckade/skippade. Omfattar riktig autentiseringsprovider/sessionlogin, API-skrivning, privata läsningar, egna/andras inlägg och trådar, överföring, moderatorer/deltagare, profiler, rolleskalering, adminblockering/återställning/radering, aktiv blockerad session, CSRF och upprepad adminprovisionering. Dashboarden och login-sidan renderades också.
- Två nya livscykeltester misslyckades under utvecklingen och hittade en Hibernate-referensdefekt vid trådradering. Explicit ordnad borttagning av inlägg och relationer korrigerade den; samma tester passerade därefter.
- Paketerad JAR startades mot PostgreSQL 18.6 på port 8081 med `blug_forum_test`. `scripts/smoke-test.ps1` passerade 124 kontroller med riktiga HTTP-anrop, cookies och CSRF-token; syntetiska data var populära, inte bara en tom forumlista.
- Livekontrollen hittade också fel status vid anonym CSRF-nekning: en ERROR-dispatch gav 401 i stället för 403. Säkerhetskonfigurationen tillåter nu intern ERROR-dispatch så originalstatus bevaras. Återkörning verifierade 403. Två tidiga smoke-fel var PowerShell-hantering av tomma bytesvar och cachade headers; båda korrigerades utan försvagade assertions.
- pg_dump/pg_restore från `blug_forum_test` till ny `blug_forum_restore_test` genomfördes utan överskrivning av befintlig databas. Radantal och fingeravtryck för varje tabells fullständiga rader matchade: users 10, forums 4, threads 6, posts 5, moderators 0, participants 2. En representativ fyrtabellsjoin verifierade återställd inläggstext, ägarskap, användarroll och blockstatus. Dessa antal gäller denna körning med syntetiska uppgifter.
- Kvarstående kontraktsblocker: lärarens endpointspecifikation har ännu inte lämnats efter att länken visat sig otillgänglig. Ingen faktisk Blug-frontend har testats. Därför hävdas inte exakt lärarkontrakt eller frontendkompatibilitet.

Feature-commits: `adae563` sessionsäkerhet (inkluderar bevarad samtidig commit `9105bfe`), `54ef3e5` medlemslivscykel, `73f7554` trådmoderering, `38472ab` adminlivscykel och `eaf1b5c` slutlig verifiering/dokumentation. De är pushade på respektive `feature/session-security`, `feature/member-lifecycle`, `feature/thread-moderation`, `feature/admin-lifecycle` och `feature/verification-docs`. Samtliga integrerades i den pushade `release`-branchen; verifieringsintegrationen är `01ccd20`, följd av denna dokumentationsuppdatering. `main` har inte pushats i detta arbete. Kontrollera aktuell release-tip med `git log -1 --oneline`.

Individuell analys – säkerhet och integritet i Blug Forum

Detta är en teknisk analys av implementationen, inte påståenden om Angelicas personliga erfarenheter eller ett intyg om juridisk efterlevnad. Projektet är en skolbackend. Blug levererar frontend separat, och dess integration är ännu inte verifierad.

Arkitektur, uppgifter och skyddsvärden

En Spring Boot-applikation erbjuder REST och en befintlig Thymeleaf-dashboard. Spring Security använder sessionscookie, BCrypt och CSRF-token. Spring Data JPA lagrar användare, forum, trådar och inlägg i PostgreSQL. Trådägarskap, moderatorkoppling och deltagarkoppling ger objektspecifika rättigheter. Globala administratörer provisioneras direkt till autentiseringens `users`-tabell. Den äldre `UserData`-modellen används inte för API-identitet.

Personuppgifter omfattar användarnamn, email, lösenordshash, publicerad text, skapandetider samt kopplingar mellan användare och innehåll. Sessions-id och driftloggar kan också vara identifierande. Fri text kan innehålla känsliga uppgifter även om applikationen inte efterfrågar dem. Tillgångar är konton, privata diskussioner, rättigheternas korrekthet, innehållets integritet och databasens tillgänglighet.

Angripare kan vara anonyma besökare, en medlem som testar andras id, en moderator som missbrukar åtkomst, en extern webbplats som försöker använda webbläsarens session och någon med stulna lösenord eller en databasdump. Angreppsytor är login/registrering, objekt-id i REST-anrop, användarstyrd JSON, cookies, dashboard-formulär, databasporten och backupfiler. En komprometterad administratör har avsiktligt stor påverkan.

Tre prioriterade risker

Bedömningen är kvalitativ för en enkel forumtjänst, inte uppmätta frekvenser.

Prioritet / risk	Sannolikhet och konsekvens	Genomförbar motåtgärd och motivering
1. Otillåten objektåtkomst och dataläckage	Hög i den tidigare koden: privata trådar returnerades utan kontroll och entiteter exponerade relationer. Stor konsekvens för privata diskussioner och kontouppgifter.	Hög genomförbarhet: central objektskontroll, DTO-svar, betrodd sessionsidentitet och regressionstester. Implementerat. Grundläggande åtkomstkontroll ger stor nytta utan ny infrastruktur.
2. Oönskade sessionstransaktioner och kontoövertagande	Medelhög: tidigare CSRF avstängt, svaga utvecklingskonton och login saknar hastighetsbegränsning. Stor konsekvens: ändrat/raderat innehåll eller administratörsmissbruk.	CSRF, BCrypt, explicit provisionering och kontroll av blockerade användare vid varje skyddad operation är implementerade till rimlig kostnad. Rate limiting och TLS vid verklig drift är kvarstående rekommendationer.
3. Dataförlust eller läckande backup	Medelhög: enkla lokala behörigheter, permanenta raderingar och inga automatiska backuper. Stor konsekvens för hela tjänsten.	Hög genomförbarhet för enkel pg_dump, skyddad förvaring och provåterläsning i isolerad databas. Instruktioner finns; slutlig verifiering dokumenteras separat. Ingen backupplattform har byggts.

Genomförda förbättringar och avvägningar

API:t binder uttryckliga requestfält och returnerar avgränsade DTO:er. Klienten kan inte välja sin roll, författare, ägare eller blockeringsstatus. Databas-id är knutet till sessionsidentiteten, så en gammal session inte får rätt till ett nytt konto med samma användarnamn. Blockering kontrolleras i databasen och gäller även en pågående session. ADMIN kontrolleras också mot aktuell databastilldelning. Ägare är trådens nuvarande ägare, och en global MODERATOR-enum ger ingen generell moderationsrätt.

CSRF är aktivt även för registrering, login och logout. Frontend måste hämta en ny token efter login/logout och bevara cookies; detta ökar integrationsarbetet men skyddar webbläsarsessioner. Inga JWT-, OAuth- eller MFA-system har lagts till. Validering begränsar storleken på fält och felsvar exponerar inte interna undantag. SQL- och säkerhetsdebugloggning är avstängd i standardinställningen.

Privata trådars metadata listas utan innehåll eller medlemsrelationer enligt den provisoriska tolkningen. Det gör forumet lätt att navigera men innebär att titlar inte får användas för hemlig information. Blockering behåller text för återställning; författarradering tar bort inläggstext medan permanent deletion tar bort databasrader. Kontoradering tar även bort ägda trådar och därmed andras inlägg i dem. Det ger enkel och förutsägbar referensintegritet, men ägarbyte bör göras först om gruppen vill bevara diskussionen.

Profilens användarnamn är oföränderligt för en enkel identitetsmodell. Lösenordsbyte avslutar aktuell session; andra egna sessioner kan fortfarande vara aktiva. Det senare är en kvarstående risk, inte en genomförd funktion för global sessionsåterkallelse. Testerna använder syntetiska uppgifter i en separat databas för att undvika skador på gruppens data.

GDPR och livscykel

Ändamål: tillhandahålla konto, deltagande i diskussioner och nödvändig moderering. Email kan användas som kontaktuppgift men tjänsten har inget emailflöde; behovet bör prövas innan verklig drift. Ingen extra profilinformation efterfrågas. Hashvärden lämnar inte API:t och email returneras endast i egen profil eller behörig administratörsoperation.

Möjlig rättslig grund måste väljas av den ansvariga organisationen. Avtal kan vara relevant för uppgifter som behövs för en begärd kontotjänst, och berättigat intresse kan behöva prövas för moderering och säkerhet. Detta är möjliga bedömningar, inte fastställda grunder för just denna tjänst. EU-kommissionen beskriver rättsliga grunder och principerna om ändamål, minimering och lagringstid.

Retention behöver fastställas före verklig användning. För skolprojektet rekommenderas syntetiska data och radering av testdata efter examination. För drift behövs en dokumenterad gräns för inaktiva konton, modereringsunderlag, loggar och backuper. Rekommenderad backupretention är 30 dagar med begränsad åtkomst; det är inte automatiserat. Blockering är inte radering: bevarad text måste omfattas av retention. Självradering tar bort konto och kopplat innehåll enligt guiden, men äldre backuper och eventuella personuppgifter i andras text behöver hanteras separat. Efter återläsning måste tidigare raderingar återappliceras.

Backup och nästa sprint

`fortsattningsguide.md` innehåller konkreta pg_dump/pg_restore-kommandon för en isolerad databas. Återläsning ska testas utan att skriva över gruppens databas. Skydda även dumpfiler eftersom de innehåller personuppgifter och hashvärden. En genomförd återläsningskontroll dokumenteras i guidens verifieringsstatus; planerade rutiner ska inte beskrivas som redan automatiserade.

Verifierat 2026-10-12: 18 automatiska tester och 124 riktiga HTTP-kontroller passerade. En dump med syntetiska data återlästes till `blug_forum_restore_test`; tabellernas samtliga radfingeravtryck och en representativ innehålls-/ägarskapsrelation matchade källan. Detta visar en fungerande lokal återläsning, inte att en daglig backuprutin, krypterad backupförvaring eller juridisk efterlevnad har etablerats. Livekontrollen hittade och verifierade också fixen som bevarar 403 vid anonym CSRF-nekning genom servlet ERROR-dispatch.

Nästa sprint bör verifiera lärarens exakta kontrakt och Blugs frontend, begränsa loginförsök, lägga till paginering för större datamängder samt granska XSS-säker rendering i frontend. Före extern drift behövs TLS, lämpliga cookie-/databasinställningar, integritetsinformation och fastställt personuppgiftsansvar. Lokala databaslösenord och `ddl-auto: update` är utvecklingsinställningar. Efterlevnad av GDPR är inte fastställd genom dessa kodändringar.